

TD&TP 1&2

Note : Pour l'équation de la complexité vous aurez soin d'expliquer la démarche suivie et de justifier les termes qui la composent.

////////////////////////////////////

ALGORITHMES ITÉRATIFS

EXERCICE 1

Etudier la complexité de la fonction somme qui calcule la somme du n premier nombre entier (en ne comptabilisant que les additions).

Fonction somme(n) -> entier Début som ← 0 pour i variant de 1 à n faire som ← som + i fpour Renvoyer som Fin	<u>Paramètres formels :</u> n : entier. Paramètre en entrée(E) <u>Variables intermédiaires :</u> Som, i : entier
--	---

EXERCICE 2

En ne comptabilisant que les **multiplications**, trouver et analyser en fonction de n l'équation de la complexité en temps de la fonction suivante :

Fonction tata(n) → Entier Début som ← 0 Pour i variant de 1 à n faire pour j variant 1 de i faire som ← som + $i*j$ finpour finpour Renvoyer som Fin	<u>Paramètres Formels :</u> n : entier. Paramètre en entrée (E) <u>Variables intermédiaires :</u> i, j, som : entier.
--	---

EXERCICE 3

En ne comptabilisant que les additions (ligne I et II) ; trouver en fonction de n l'équation de la complexité en temps de l'algorithme suivant :

Procédure tata(n, s) Début m ← 0 pour i variant de 1 à n faire m ← m + i (I) finpour s ← 0 pour i variant de 1 à $m*m$ faire s ← s + i (II) fpour Fin	<u>Paramètres Formels :</u> n : entier. Paramètre en entrée (E) s : entier. Paramètre en sortie (S) <u>Variables intermédiaires :</u> i, m : entier
--	---

EXERCICE 4

En ne comptabilisant que les comparaisons, étudier la complexité de la fonction qui recherche de manière séquentielle un élément dans un **tableau trié** composés de n éléments.

Fonction RecheSéquenTabTrié(n, X, tab) → booléen

Début
 si $X > \text{tab}[n]$ alors
 Renvoyer faux
 sinon
 $i \leftarrow 1$
 tant que $\text{tab}[i] < X$ faire
 $i \leftarrow i + 1$
 ftq
 Renvoyer $\text{tab}[i]=X$
 fsi

Fin

////////////////////////////////////

ALGORITHMES RÉCURSIFS

EXERCICE 5

En ne comptabilisant que les exécutions de l'affichage produit par la procédure **écrire**, trouver et analyser en fonction de n l'équation de la complexité en temps de l'algorithme suivant :

Procédure affiche_tab(n, tab) Début si($n>0$) alors écrire($\text{tab}[n]$) affiche_tab($n-1, \text{tab}$) finsi Fin	<u>Paramètres Formels :</u> n : entier (E) tab : tableau (E)
--	---

EXERCICE 6

Calcul d'une exponentielle à exposant entier ≥ 0 .

Soit à calculer x^n pour x quelconque et n entier ≥ 0 .

a) La forme naturelle du calcul consiste à remarquer que: $x^i = x \cdot x^{i-1}$ pour $i > 0$ et $x^0=1$.

Quelle est la complexité de cet algorithme?

b) Exponentiation indienne:

Posant: $f(x, n) = x^n$, on remarque que:

$f(x, 2p) = f(x^2, p)$ pour $p > 0$ (on suppose que $f(x, 0)=1$)

$f(x, 2p+1) = x \cdot f(x, 2p)$

Calculer la complexité de cette solution.

c) Qu'en concluez-vous quant au choix d'un algorithme pour calculer x^n ?

EXERCICE 7

En ne comptabilisant que les exécutions de l'affichage produit par la procédure **écrire**; étudier en fonction de **n** la complexité en temps de la procédure **tita** (pour faciliter la résolution de l'équation de la complexité on suppose que **n** est une puissance de 2).

<pre> procédure tita(n) Début si n > 1 alors m ← n / 2 pour i variant de 1 à n faire écrire('a+') fpour tita(m); tita(m); si non écrire('a-') finsi Fin </pre>	<u>Paramètres Formels :</u> n : entier(E) <u>Variables intermédiaires :</u> m : entier
---	--

EXERCICE 8

En ne comptabilisant que les *divisions (ligne I)*, trouver en fonction de **n**, l'équation de la complexité en temps de la fonction **tata** ci-dessous (Pour faciliter la résolution de l'équation de la complexité on suppose que **n** est une puissance de 2).

<pre> Fonction tata(n, m) → entier Début si n=0 alors renvoyer m sinon (I) p ← n / 2 renvoyer tata(p, p) + tata(p, m) finsi Fin </pre>	<u>Paramètres Formels</u> n, m : entier (E) <u>Variables intermédiaires</u> p : entier
--	--

EXERCICE 9

En ne comptabilisant que les *déplacements* lors de l'appel de la procédure **écrire**, trouver et analyser en fonction de **n** l'équation de la complexité en temps de l'algorithme suivant :

<pre> procédure tour_hanoi(n, A, B, C) Début si n = 1 alors écrire('Déplacement de ', A, 'vers ', C) si non tour_hanoi(n-1, A, C, B) écrire('Déplacement de ', A, 'vers ', C) tour_hanoi(n-1, B, A, C) finsi Fin </pre>	<u>Paramètres formels :</u> n : Entier (E) A, B, C : Disque (E)
---	---

TP

1) Implémenter en langage C les algorithmes de tris suivants :

- Tri Par Sélection
- Tri Par Insertion
- Tri Fusion
- Tri Rapide

2) Tester vos programmes sur les tableaux d'entiers suivants :

- a) Génération aléatoire d'un tableau de **n** éléments dont les éléments sont compris entre 100 et 200.
- b) Génération d'un tableau de **n** éléments à partir d'un fichier texte en entrée dont la structure est :
NNb (nombre de nombre à trier)
VNi (valeur de chaque nombre).

3) En utilisant GNUPLLOT, étudier graphiquement le temps d'exécution des différents algorithmes de tri.